

智能进化：AI Agent 重塑 IT 生产力

从 Copilot 到 Autopilot，开启企业提效新范式



Darren Cui

HKHPC

12 Dec 2025

IT 生产模式的“内卷”困境

- 需求堆积、交付周期冗长，排期越来越难。
- 大量重复性工作（CRUD、告警处理、脚本维护）消耗研发人力。
- 跨部门沟通成本高，信息孤岛明显，文档分散。
- 技术栈快速迭代，学习与迁移成本不断上升。
- 核心问题：如何从“堆人力”转向“堆算力 + 智能”的新生产方式？

概念重构：LLM、Copilot 与 AI Agent

- **LLM**：会“思考”但不直接执行——主要提供语言理解与生成。
- **Copilot**：人主导、AI 辅助——补全代码、给建议，但不闭环。
- **AI Agent**：目标驱动的自主智能体，具备：
 - 感知 (Perception) ——读取代码库、日志、文档。
 - 规划 (Planning) ——拆解任务、生成多步计划。
 - 记忆 (Memory) ——跨对话、跨文件保持上下文。
 - 工具使用 (Tool Use) ——调用 CLI、API、脚本，自动执行闭环。
- **标准 Agent Pipeline**：LLM + Planning + Memory + Tools。

从问答机器人到垂直领域机器人

- 最开始的使用方式：普通问答机器人，类似“智能搜索 + 聊天”。
- 进阶：在 Poe 等平台设置语料库，构建特定垂直领域机器人。
- 感受：比搜索引擎更懂上下文，但仍然停留在“我问它答”的阶段。
- 引子：AI 是否只能做问答？能否真正作为 **Agent** 主动执行任务？

AI + Markdown: 轻量级画图思路

■ 结合经验与 AI：知道 Markdown 支持 mermaid，就能把“画图”脚本化。

■ 典型 workflow：

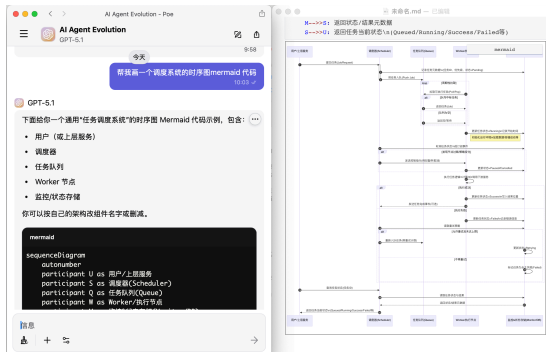
1. 在 .md 文本编辑器中输入 ````mermaid` 然后回车。
2. 让 AI 生成 mermaid 代码，复制粘贴到代码块中。
3. 编辑器自动渲染流程图、时序图、甘特图等。

■ 本质：经验（工具语法）+ AI，实现半自动化、极大提升效率。

Mermaid 轻量画图：示例（一）



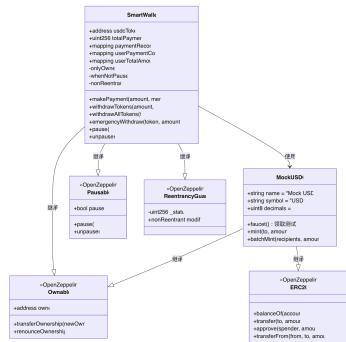
项目进度甘特图



应用系统时序图

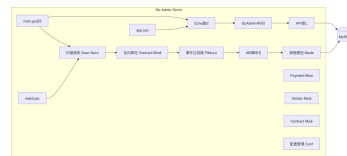
Mermaid 轻量画图：示例（二）

智能合约架构



mermaid-example1.png

后端服务架构



安全防护层级



mermaid-example2.png



mermaid-example3.png

从「把问题丢给 AI」到「把流程丢给 AI」

- 写代码时：把 error 文本、log、栈追踪直接丢给问答机器人。
- 学编程语言：几乎不再查大部头用户手册；StackOverflow 的知识，被 AI 过滤重组后呈现。
- 但：问答机器人 **AI Agent**——它只在你“问”的那一步出现。
- 真正的 Agent：能理解你的目标，主动规划命令和操作序列，自动执行。

我与 AI Agent 的第一次“正面相遇”

- 10 月份加入团队，接手同事已搭好的代码。
- 起初模式：复制粘贴代码和报错到 GPT 5.1 聊天框，一步步跑通。
- 同事提醒：VS Code 自带 Agent 聊天窗口，可以直接在项目上下文中工作。
- 打开 Agent 模式 = 打开新世界的大门：
 - 它能直接读项目文件、运行命令、改代码。
 - 但也遇到瓶颈：每个操作都要点 approve，不能并行多个任务窗口。

命令行里的新世界：Claude Code & 终端交互

- 发现 Claude Code / Gemini CLI / Copilot CLI / Codex CLI 等主流 SWE AI Agent。
- 在命令行直接和整个项目“对话”，让终端成为 AI 的“操控台”：
 - 扫描仓库结构、搜索文件、生成/修改代码。
 - 运行测试、分析日志、执行部署脚本。
- 对于需要并行执行多个任务的场景（多个 Shell 会话），命令行 Agent 尤其高效。

模型选择界面示例

```
European_XFEL_LaTeX_Presentation — claude TERM_PROGRAM=Apple...

-----
Select model
Switch between Claude models. Applies to this session and
future Claude Code sessions. For other/previous model
names, specify with --model.

1. Default (recommended)      Use the default
                               model (currently
                               Sonnet 4.5) · $3/$15
                               per Mtok
2. Opus                        Opus 4.5 · Most
                               capable for complex
                               work · $5/$25 per
                               Mtok
3. Haiku                       Haiku 4.5 · Fastest
                               for quick answers ·
                               $1/$5 per Mtok
> 4. doubao-seed-code-preview-latest Custom model ✓

Enter to confirm · Esc to exit
```

```
European_XFEL_LaTeX_Presentation — node ~/.npm-global/bin/copilot...

✖ You must be logged in to send messages. Please run /login

Select Model

Choose the AI model to use for Copilot CLI. The selected
model will be persisted and used for future sessions.

Some models are not available due to configured policy.

> 1. Claude Sonnet 4.5 (1x) (default) (current)
2. Claude Sonnet 4 (1x)
3. GPT-5 (1x)
4. GPT-5.1-Codex-Mini (0.33x)
5. GPT-5.1-Codex (1x)
6. Cancel (Esc)

Confirm with number keys or ↑↓ keys and Enter, Cancel with
Esc
```

安装与地区限制：现实中的“隐形门槛”

- 安装本身几乎是“一键式”的：
 - 例如 Gemini CLI：在 macOS/Linux 上 `brew install gemini-cli`。
- 但在中国大陆和香港地区：
 - Claude Code、Codex、Gemini CLI 等受到区域限制。
 - Copilot CLI 相对友好，在大部分场景下可以正常使用。
- Google 侧限制尤其严格，即使 VPN 全局代理，也经常被风控阻断。

Gemini CLI 的“曲线救国”方案

- 在 <https://console.cloud.google.com/> 注册 Google Cloud 账号，创建项目。
- 获取 Project ID，设置环境变量，例如：

```
export GOOGLE_CLOUD_PROJECT="YOUR_PROJECT_ID_XXX" >> ~/.bashrc  
source ~/.bashrc
```
- 然后可以用 Gmail 账号登录 Gemini CLI，通常可使用 Gemini 2.5 Pro。
- 可以尝试 `/settings` 命令打开 Preview Features，理论上可升级到 Gemini 3 Pro。

用 Claude Code 接入国产模型（豆包示例）

- 主流观点：当前编码能力最强的国产 coding 模型包括：
 - 字节跳动 Doubao-seed-code（常被认为仅次于 Claude Sonnet 4.5）。
 - 智谱 GLM 4.6 等。
- 在火山引擎购买“方舟 Coding Plan”，获取 Ark API Key。
- 安装 Claude Code 后，通过配置环境变量接入豆包模型：

```
vim ~/.claude/settings.json
```

```
{  
  "env": {  
    "ANTHROPIC_AUTH_TOKEN": "<ARK_API_KEY>",  
    "ANTHROPIC_BASE_URL": "https://ark.cn-beijing.volces.com/api/coding",  
    "ANTHROPIC_MODEL": "<Model>"  
  }  
}
```

Copilot CLI + Claude: 不用“魔法”的组合拳

- 问题：我既想用 Claude Code 作为 AI Agent，又想享受 Claude Sonnet 4.5 的推理能力。
- 现实：Claude Code Web 在国内地区不可用，但 Copilot CLI 可以用。
- 解法：通过 Copilot CLI 接入 Claude 模型，形成组合：
 - Copilot CLI 负责本地终端集成与命令行体验。
 - 后端模型可以切换到 GPT 5.1、Claude Sonnet 4.5 等。
- 结果：无需额外“魔法”，即可在 Terminal 中享受 Claude 级别的 Agent 能力。

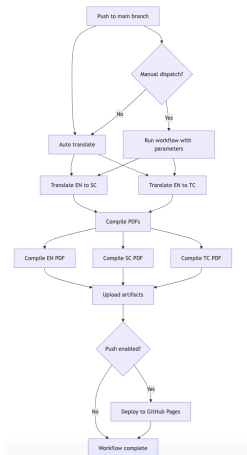
案例：AI 写长文 Report 的痛点

- 场景：基于既有白皮书迭代新版，同时要出英文、繁中、简中三种版本。
- 痛点 1：PDF 理解能力有限、Token 消耗高。
- 痛点 2：十万字级长文本翻译，AI 容易中断、风格不统一。
- 痛点 3：LaTeX 排版 & Debug 成本高，人工改格式很耗时。
- 痛点 4：一处内容更新，需要手动在三种语言版本中同步，非常繁琐。

尝试的解决方案：从 PDF 到自动发布

- 第一步：将 PDF 转为纯文本（如 .md），再喂给大模型：
 - 文本比 PDF 更容易、更便宜让模型理解。
 - Token 消耗大约可降到 PDF 的 $1/2 \sim 1/20$ 。
- 第二步：在原始文本上“覆盖式”编辑，而不是单独生成新文档。
- 第三步：让 AI Agent 直接操作 LaTeX 源码，协助排版与 Debug。
- 第四步：用大模型 API + 脚本 + GitHub Actions：
 - 自动翻译多语言版本。
 - 自动 LaTeX 编译、网站构建与部署。
 - 自己几乎不需要手写代码即可完成。

多语言发布 + CI/CD 自动化流程与成品示例



白皮书源稿章节示例



白皮书排版成品页

思考一：给 AI “降维” —— 文本 vs PDF

■ 文本文件 (.md/.tex):

- 没有复杂格式和 OCR 噪音，结构更清晰。
- 模型更容易抽取章节结构、公式、列表。

■ PDF:

- 文本型 PDF 还好，扫描版需依赖 OCR，容易出错。
- 上传到 Poe.com 时，效果取决于模型和排版质量。

■ 记忆模式:

- 上下文记忆：仅当次对话有效。
- 训练记忆：模型固化的知识，无法被对话直接更新。
- 目前没有“长期个人记忆”，所以要靠脚本和仓库来“留住”成果。

思考二：脚本化 = 更适合 AI Agent 的场景

- Word 文档：难以完全脚本化；格式操作对机器不友好。
- LaTeX：从内容、排版到编译，全流程都可用命令行控制，天然适合 Agent。
- 类推：做 PPT 是否也能脚本化？
 - 本次分享的 PPT 就是用 LaTeX + AI Agent 完成的。
 - 我主要专注内容；版式、格式、细节交给 AI。
- 结论：能脚本化的任务，更容易被 AI Agent 高效接管。

思考三：架构思维 & Domain Knowledge 的价值

- 工程实现需要大量“用户手册 + 实战经验”，例如：
 - CI/CD Workflow、YAML 语法、部署策略等。
- 外行人描述需求：只会说“帮我做个网站 / 小程序”。
- 内行拆解任务：前端、后端、测试、运维、日志、监控.....每一步可交给 Agent。
- 设计师知道“毛玻璃特效”，PPT 设计师知道“分镜结构”：
 - 这些专业词汇让你能更精确地驱动 AI。
- 所以：**AI 时代，Domain Knowledge 不会被取代，反而更重要。**

Prompt “套娃”：让 AI 帮你写 Prompt

- 原始需求：“请帮我写一个网页，加入工业风金属元素，高端风格。”
- 套娃思路：
 - 把这句简单需求喂给 AI。
 - 告诉它：你是外行，希望它以“硅谷科技公司顶级 UI + 前端工程师”的身份，帮你升级这个 Prompt。
- 结果：AI 生成了一个极其专业的 Prompt：
 - 描述工业风视觉语言、材质（拉丝金属、磨砂金属、冷灰色系）。
 - 说明栅格系统、信息架构、动效细节、HTML/CSS 结构建议。
- 视觉落地：AI 直接生成高保真网页 UI（见后续示例），已经可以生产可交付稿。
- 小结：一次简单的“套娃”，你就获得了接近 Apple/Tesla 级别的设计说明书。

Prompt “套娃”：AI 网页示例（1/2）



Prompt “套娃”：AI 网页示例（2/2）

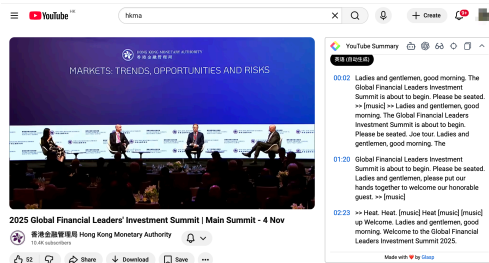


个人知识库：从博主到“专属智能体”

- 场景：你很喜欢一个博主 / 技术宅 / 影评 / 投研分析，但他的视频太多、没时间刷完。
- 做法：
 - 用 <http://glasp.co/> 抓取视频字幕，一键复制文本。
 - 把这些字幕整理成语料库，交给大模型进行分析和问答。
- 效果：
 - 为某个博主“生成一个智能体”，可以随时询问他的观点与方法论。
 - [glasp-subtitle.png](#) 可作为实际操作界面示例截图。



glasp logo



glasp subtitle

NotebookLM：从语料到播客与幻灯片

■ NotebookLM 是一个面向个人知识管理的神器：

■ 可以根据上传的语料（文本、PDF、视频链接）生成播客、报告、幻灯片。

■ 使用技巧：

■ 生成的幻灯片通常以 PDF 形式导出，审美不错（类似 Nano Banana 风格）。

■ 想继续编辑，可用 WPS 将 PDF 转成 .pptx，再进行微调。

■ 我尤其喜欢播客功能：

■ 跑步、做家务、躺着休息时都能“听”自己的知识库。

■ 对于自媒体创作者，这是极大提升生产力的工具。

二创时代：AI 让“模仿 + 创新”更高效

- 现实中：二次创作者在小红书、B 站经常比原作者获得更多流量。
 - 小米 SU7 被认为向保时捷学习外观，但大众非常买账。
- 过去没有 AI 时，二创依然门槛很高：剪辑、撰稿、排版都很耗时。
- AI 时代：二创效率至少提高一个数量级：
 - 快速整理原始素材、提炼观点、生成新表达。
 - 辅助生成封面、标题、文案、脚本。
- 摄影 / 影像创作延伸：
 - “摄影大赛”升级“AI 创作大赛”？——常见创作方式：给两张照片作为首帧和尾帧，就能生成一段视频。

Claude CLI 常用指令与 workflows (补充)

■ 常见用法示例：

- `claude`：普通对话模式。
- `claude code`：侧重代码理解、重构、测试生成。
- `claude chat .`：在当前项目目录中开启上下文对话。

■ 授权策略：

- 默认每次文件修改、命令执行前都会请求确认 (approve)。
- 可以使用类似 `claude --dangerously-skip-permissions` 之类的参数，在“完全信任”的仓库中自动执行命令。
- 建议：仅在熟悉且可随时回滚的项目中使用“dangerously”选项。

上下文记忆与 CLAUDE.md

■ 上下文记忆如何把握？

- 把长任务拆成若干可在单次对话内完成的步骤。
- 经常让 Agent 用“要点总结”压缩当前进展，减少 Token 压力。

■ CLAUDE.md 是什么？

- 放在仓库根目录的说明文件，类似项目级“使用说明 + 约束”。
- 内容包括：项目背景、技术栈、目录结构、代码风格、约定、禁用操作等。
- 好处：
 - ▶ 每次对话都可自动被读取，减少重复解释，间接节约 Token。
 - ▶ 保证不同对话中行为一致，比如统一日志风格、错误处理策略。

■ 对应 Gemini 等：

- 也可以通过配置文件或系统 Prompt 方式，扮演“项目说明书”的角色。

/subagent、/commands、skills 与 MCP

/subagent:

- 在同一个会话中创建多个“子代理”，例如“测试工程师”、“安全审计员”、“性能优化师”。
- 适合复杂任务的多角色协作，例如大型重构或安全评估。

/commands & skills:

- 定义常用命令/技能的快捷方式，如“一键生成测试用例”、“一键整理 Changelog”。
- 相当于给 Agent 配备一套“宏命令”工具栏。

MCP (Model Context Protocol):

- 通过 MCP, Agent 可以安全地访问更多外部系统:
 - 内部知识库、数据库、Issue Tracker、监控告警系统等。
- 示例场景:
 - 从监控系统取最近 1 小时 500 错误日志，自动分析根因。
 - 读取 Jira 任务列表，为每个任务生成实现建议与测试清单。

软件开发（DevAgent）

自动需求理解：

- 读取 PRD / 设计文档，生成架构图、数据库设计、API 定义草案。

代码生成与重构：

- 输入“生成登录模块” → 输出全栈代码 + 单元测试雏形。

- 对遗留代码自动加注释，给出重构建议与潜在风险提示。

自动化测试：

- 识别代码变更 → 生成测试用例 → 自动跑回归测试 → 尝试简单 Bug 修复。

智能运维（AIOps Agent）与 Data Agent

智能运维（AIOps Agent）：

- 故障自愈：告警 → 日志分析 → 根因定位 → 自动执行重启 / 回滚 / 扩容。
- 资源优化：分析云资源使用情况，给出降本建议与变更脚本。
- 安全巡检：定期扫描漏洞，模拟攻击路径，生成风险报告。

数据分析与 BI（Data Agent）：

- Text-to-SQL：自然语言提问 → 自动生成 SQL → 查询 → 图表展示。
- 趋势洞察：自动产出周报 / 月报，发现异常点并给出预警说明。

项目管理（PM Agent）与更多待探索场景

■ 项目管理（PM Agent）：

- 会议自动化：转录 → 生成会议纪要 → 提炼 Action → 自动创建任务。
- 知识库问答：整合内部文档，变成可检索、可追问的团队知识中枢。

■ 其他更多场景，仍在快速演化：

- 法务、合规、风控、运营、客户支持等。
- 建议：不要 FOMO，选取自己最痛的 1~2 个场景先做深。

安全与风控：常见风险点

代充风险：

- 别人帮你代充账号，往往需要验证码，相当于获得了你的 Google 账号“权限通行证”。
- 涉及银行卡、隐私信息，甚至存在海外黑钱、洗钱风险。

使用别人的 API Key：

- 风险包括：被滥用、被风控封号、欠费难以追责。
- 内容是否会被第三方记录与分析？需要提前想清楚。

使用 Router / 聚合代理：

- 等于让你的请求经过别人服务器中转。
- 问题是：你的内容，你是否愿意给对方看？

安全实践建议

- 不要在聊天窗口直接粘贴私钥、API Key、数据库密码。
- 建议用环境变量形式管理：
 - 如 `export OPENAI_API_KEY=...`，在 Shell 中设置。
- 账户充值与订阅：
 - 尽量使用官方渠道，避免不明来源的“便宜代充”。
- 数据最小暴露原则：
 - 只上传为完成任务必须的最小数据集。
 - 对敏感字段（用户信息、财务数据）尽量脱敏后再交给 AI。

我的原则：把“一天的工作”迁移到 AI 场景

- 雷军的愿景：让用户“把一天所有的事情都在小米手机上完成”。
- 类似地，斯坦福新课《CS146S: The Modern Software Developer》：
 - 课程主旨：教你在几乎不写代码的情况下进行软件开发。
- 我的迁移目标：
 - 把尽可能多的工作，迁移到“与 AI 对话”的形式完成。
 - 从写代码、改文档，到排版、测试、部署、汇报，都尽量交由 Agent。

模型选择：不迷信“西方月亮更圆”

■ 国产模型近两年进步巨大，性价比极高。

■ 例子：

■ Doubao-seed-code 在编码任务上，主观体验优于 Grok fast 1 / GPT 5 mini。

■ 在某些中文视频生成场景，可灵 AI 生成的人物对话场景明显优于 Veo-3。

■ 结论：选适合场景 + 适合预算的组合即可，无需盲目 FOMO 海外产品。

总结与行动建议

从 Copilot 到 Autopilot:

- 先让 AI 帮你“改写一段代码 / 一页文档”，再让它“改一个项目 / 一本书 / 一套工作流”。

接下来可以做的三件事:

- 选一个可脚本化的痛点任务，尝试交给 Agent 做闭环。
- 建一个小型个人知识库（白皮书、收藏文章、喜欢的博主字幕）。

最后想说:

- AI 不会取代有架构思维和 Domain Knowledge 的人。
- 拥抱变化，用好 AI 相比不会使用 AI 会带来显著的生产力差距。